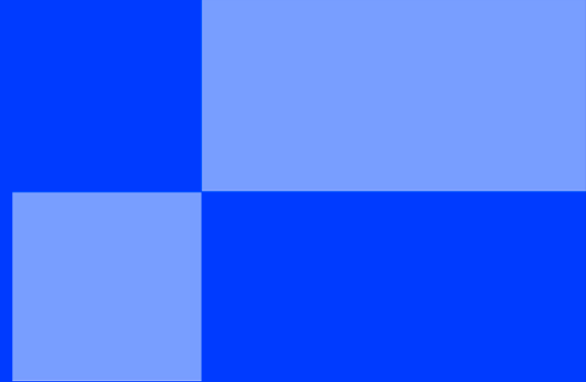




ATELIER ANALYSE - PAYS HÔTES ET MÉDAILLES, LE VERDICT

< WINTER CAMP 2026 />

ATELIER ANALYSE - PAYS HÔTES ET MÉDAILLES, LE VERDICT

Bienvenue dans ce deuxième atelier ! Nous allons maintenant utiliser les compétences du `starter.md` pour répondre à une question plus complexe : **Les pays obtiennent-ils plus de médailles quand ils organisent les Jeux Olympiques d'hiver ?**

Pour cela, nous allons devoir manipuler nos données de manière plus avancée.

Étape 1 : Regrouper sur Plusieurs Colonnes

Explication

Dans l'atelier précédent, nous avons regroupé les données sur une seule colonne. Pour une analyse plus fine, nous pouvons regrouper sur **plusieurs colonnes** en même temps en passant une liste de noms de colonnes à la fonction `groupby()`. L'ordre des colonnes dans la liste est important car il définit la hiérarchie des groupes.

Exemple

Imaginons un DataFrame de ventes de produits.

```
# Regrouper sur une liste de colonnes
ventes.groupby(['produit', 'magasin', 'mois']).size()
```

Le résultat nous donnera le nombre de ventes pour chaque combinaison unique (e.g., 2 Laptops vendus à Paris en Janvier).

Expérimentation

✓ Bloc de code :

- Reprenez le DataFrame `df` du `starter` contenant les médailles des JO.
- Regroupez les données par `country`, `host` et `year` pour compter le nombre de médailles pour chaque participation d'un pays.
- Utilisez `.size()` pour faire le décompte.
- Transformez le résultat en DataFrame avec `.reset_index(name="medals_count")`.
- Stockez le tout dans une nouvelle variable `df_grouped`.

✓ **Bloc de Markdown :**

- Affichez votre DataFrame `df_grouped`. Que représente chaque ligne ?



- ✓ La fonction `.groupby()` accepte une liste de chaînes de caractères : `['col1', 'col2', ...]`.

Étape 2 : Calculer une Moyenne sur des Groupes

Explication

Une fois les données regroupées, `.size()` est utile pour compter les lignes, mais nous pouvons faire bien plus ! En sélectionnant une colonne après le `groupby()`, nous pouvons appliquer des calculs plus spécifiques comme `.mean()` (moyenne), `.sum()` (somme), ou `.max()` (maximum).

Exemple

Avec un DataFrame de notes d'étudiants, pour calculer la moyenne de *chaque étudiant*, on regroupe par `etudiant`, on sélectionne la colonne `note`, puis on applique `.mean()`:

```
# Calculer la moyenne des notes par étudiant
notes.groupby('etudiant')['note'].mean()
```

Le résultat montrera que la moyenne d'Alice est 13.5 et celle de Bob est 18.

Expérimentation

✓ **Bloc de code :**

- À partir de votre DataFrame `df_grouped` de l'étape précédente, regroupez cette fois par `country` et `host`.
- Calculez la moyenne de la colonne `medals_count` pour chaque groupe.
- Stockez le résultat dans un DataFrame nommé `df_avg_medals` (n'oubliez pas d'utiliser `.reset_index()`).

✓ **Bloc de Markdown :**

- Affichez `df_avg_medals`. Que représentent les colonnes `host=True` et `host=False` ?



✓ La structure est `df.groupby(...)[ 'colonne_a_calculer' ].mean()`.

Étape 3 : Pivoter une Table

Explication

Notre `df_avg_medals` est bien, mais pour comparer facilement les performances, nous préférierions avoir une seule ligne par pays et deux colonnes : une pour la moyenne des médailles en tant qu'hôte, et une autre en tant que non-hôte.

Nous allons utiliser la fonction `pivot_table()` qui est conçue exactement pour ça. Elle nous permet de "pivoter" une colonne pour que ses valeurs deviennent les nouvelles colonnes du DataFrame.

Exemple

Imaginons un DataFrame des notes moyennes par semestre que l'on peut pivoter pour avoir une ligne par étudiant et une colonne par semestre :

```
# Pivoter le DataFrame
notes.pivot_table(index='etudiant', columns='semestre', values='moyenne')
```

Expérimentation

✓ Bloc de code :

- Utilisez `.pivot_table()` sur votre DataFrame `df_avg_medals`.
- Utilisez `index='country'` pour que chaque pays devienne une ligne.
- Utilisez `columns='host'` pour que les valeurs `True` et `False` deviennent les nouvelles colonnes.
- Utilisez `values='medals_count'` (ou le nom que vous avez donné à votre colonne de moyenne) pour remplir la table.
- Stockez le résultat dans un DataFrame `df_pivot`.
- **Important** : certaines analyses ne sont valides que si nous avons les deux points de comparaison. Utilisez `.dropna()` sur votre `df_pivot` pour ne garder que les pays qui ont organisé au moins une fois ET participé en tant que non-hôte.

✓ Bloc de Markdown :

- Affichez votre `df_pivot` après le `.dropna()`. Combien de pays peut-on comparer équitablement ?

Étape 4 : Créer une Nouvelle Colonne

Explication

Maintenant que nos données sont bien structurées, nous pouvons enrichir notre analyse en créant une nouvelle colonne. Par exemple, nous pouvons calculer le pourcentage d'amélioration du nombre de médailles. La formule du pourcentage d'amélioration est :

```
((valeur_finale - valeur_initiale) / valeur_initiale) * 100
```

Exemple

Si un DataFrame `df` a deux colonnes `ventes_annee_1` et `ventes_annee_2`, on peut créer une colonne `progression` comme ceci :

```
# Créer une nouvelle colonne
df['progression'] = ((df['ventes_annee_2'] - df['ventes_annee_1']) / df['ventes_annee_1'])
* 100
```

Expérimentation

✓ Bloc de code :

- Dans votre `df_pivot` nettoyé, créez une nouvelle colonne nommée `boost`.
- Dans cette colonne, calculez le pourcentage d'amélioration entre la moyenne de médailles en tant que non-hôte (`False`) et la moyenne en tant qu'hôte (`True`).
- Triez votre DataFrame en fonction de cette colonne `boost` avec `.sort_values('boost', ascending=False)`.

✓ Bloc de Markdown :

- Affichez les 5 premiers pays de votre DataFrame trié. Quel est le pays avec le plus grand "boost" ?



- ✓ Les colonnes après le pivot se nomment `True` et `False`. Vous pouvez les utiliser directement dans vos calculs : `df[True]` et `df[False]`.

Étape 5 : Visualisation Avancée

Explication

Un graphique est souvent la meilleure façon de présenter une conclusion. Nous allons créer un diagramme en barres horizontales (`plt.barh`) pour visualiser le `boost` de chaque pays. Pour le rendre encore plus lisible, nous allons colorer les barres différemment si le boost est positif ou négatif.

Exemple

Imaginons un DataFrame `df` avec une colonne `pays` et une colonne `score`. On peut créer une liste de couleurs avant de faire le graphique. Cette technique de "list comprehension" est très puissante en Python.

```
# Créer une liste de couleurs : 'g' pour vert si score > 0, 'r' pour rouge sinon
couleurs = ['g' if s > 0 else 'r' for s in df['score']]

# Créer le graphique en utilisant cette liste
plt.barh(df['pays'], df['score'], color=couleurs)
plt.title("Scores par pays")
plt.show()
```

Expérimentation

✓ Bloc de code :

- Prenez votre DataFrame `df_pivot` final, trié par `boost` (pour un graphique plus lisible, triez par ordre croissant cette fois).
- Créez une liste de couleurs conditionnelle basée sur la colonne `boost`.
- Créez un graphique en barres horizontales (`plt.barh`) avec les noms des pays et leur `boost`.
- Utilisez votre liste de couleurs avec le paramètre `color`.
- Ajoutez des titres et des labels pour rendre votre graphique compréhensible.

✓ Bloc de Markdown :

- Décrivez en une phrase ce que votre graphique final montre. A-t-on répondu à notre question de départ ?

v1

{EPITECH}